



Mini-Project: Modified SAP-1

Project Overview

In this project, you will design and implement a modified SAP-1 (Simple-As-Possible-1) computer **using Verilog HDL**. The goal is to build a simple processor from scratch and extend its functionality by adding new arithmetic operations.

You are expected to:

- Design the SAP-1 architecture in Verilog
- Implement the fetch–decode–execute cycle
- Extend ALU and instruction set with:
 - Multiplication (MUL)
 - Division (DIV)

Summary of SAP-1

For more details, please refer to: <https://karenok.github.io/SAP-1-Computer>

SAP-1 (Simple-As-Possible-1) is a minimal educational computer architecture designed to demonstrate the internal operation of a basic CPU

A standard SAP-1 system consists of the following ten main components:

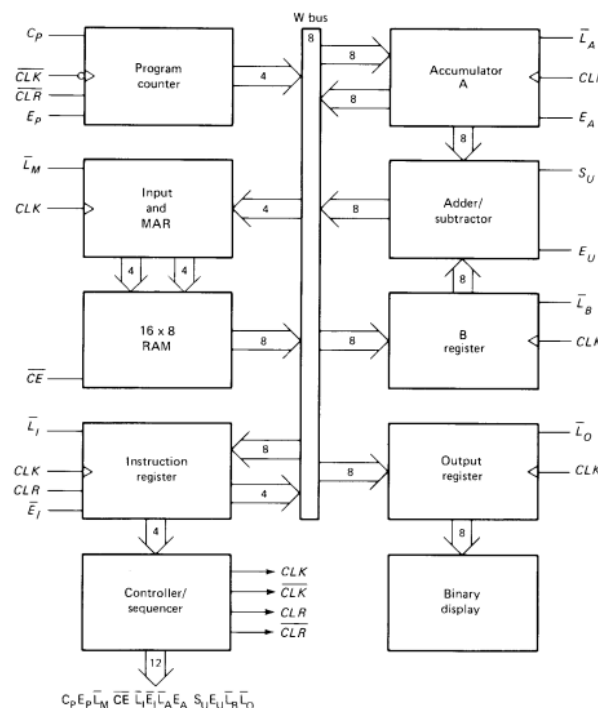


Fig. 10-1 SAP-1 architecture.

1. Program Counter (PC)
 - Holds the address of the next instruction to be executed
 - Automatically increments after each fetch cycle
2. Memory Address Register (MAR)
 - Stores the memory address that is currently being accessed
 - Sends this address to RAM
3. Random Access Memory (RAM)
 - Stores both instructions and data
 - Typically 16 locations $\times$ 8 bits in SAP-1
 - Outputs data based on the address from MAR
4. Instruction Register (IR)
 - Holds the current instruction fetched from memory
 - Splits into:
 - Opcode (operation)
 - Operand (address/data reference)
5. Control Unit / Controller-Sequencer
 - Decodes the instruction and generates control signals
 - Controls the timing and coordination of all components using an FSM
6. Accumulator (A Register)
 - Main working register for arithmetic operations
 - Stores intermediate and final results from the ALU
7. B Register
 - Temporary register used to hold the second operand
 - Feeds data into the ALU alongside the accumulator
8. Arithmetic Logic Unit (ALU)

Performs arithmetic operations, including:

 - Addition
 - Subtraction

In this project, it is extended to include:

 - Multiplication
 - Division
9. Output Register
 - Stores the result to be displayed or observed
 - Typically connected to LEDs or output ports in simulation
10. Bus System (Data Bus)
 - A shared communication pathway connecting all components
 - Transfers data between registers, memory, and ALU
 - Only one component can drive the bus at a time (controlled by control signals)

Project Task:

Task 1: Implement Core Components. Your design should include, at a minimum:

- Program Counter (PC)
- Memory Address Register (MAR)
- RAM (e.g., 16×8)
- Instruction Register (IR)
- Accumulator (A Register)
- B Register
- Arithmetic Logic Unit (ALU)
- Output Register
- Control Unit (FSM-based)
- Shared data bus or equivalent multiplexing system

Task 2: Extend the support:

- Multiplication (MUL)
- Division (DIV)

Implementation Method(Required in **ALU module only**):

- You must implement Multiplication and Division logic from scratch (Don't use * or /).

You must clearly define:

- Output behavior
- Handling of special cases (e.g., division by zero)

Task 3: Instruction Set Design.

Define your own instruction set, including at least:

Instruction	Description
LDA	Load memory value into A
ADD	Add memory value to A
SUB	Subtract the memory value from A
MUL	Multiply A by the memory value
DIV	Divide A by the memory value
OUT	Output value of A
HLT	Halt execution

- You are free to define your own opcode format
- Clearly document instruction encoding in your report

Requirement:

- Use **Verilog HDL only** (no VHDL)
- Fully functional ALU with MUL and DIV
- Multiplication and Division operations of ALU must be implemented from scratch (Don't use * or /)
- If your implementation is based on existing resources (including resources provided). Please cite them in documents.

Submission:

Deadline: 19th May 2026; 23:59

1. Source Code (Note: Submit only the necessary file. E.g., you shouldn't submit the entire project folder):

- Verilog source code (.v)
- Top-level module
- Testbench file

2. Report: Your report should include.

- System architecture and datapath design
- Instruction set and opcode format
- ALU design (including MUL/DIV implementation)
- Control unit (FSM design)
- Simulation results (waveforms and explanation)
- Design decisions and challenges

3. Presentation Slide.

4. Video presentation including video demo.